



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2133 — Estructuras de Datos y Algoritmos 1'2026

Interrogación 1

30 de abril de 2026

Tiempo: 2 horas

Condiciones de entrega: Se deben entregar 3 de las 4 preguntas.

Forma de Entrega: La entrega es a través de Canvas. Al final de la prueba tienen 10 minutos para subir la imagen de la prueba. En formato vertical, cada pregunta por separado.

Dispositivo: Para digitalizar la prueba se permite el uso de UN dispositivo (teléfono, tablet) el cual debe estar guardado y en silencio. En el caso de ir al baño el dispositivo debe dejarse adelante en la mesa con los ayudantes.

Torpedo: Se permite una hoja tamaño oficio escrita a mano.

Evaluación: Cada pregunta tiene 6 puntos (+1 punto base). La nota es el promedio de las 3 preguntas entregadas. La nota de la interrogación es el promedio de las 3 preguntas entregadas

Hint: Lea todas las preguntas antes de iniciar las respuestas.

1. Dicionarios

2. Colas con prioridades

Ud. cuenta con un computador cuya CPU solo puede atender a un proceso a la vez y lo hace en intervalos (slots) fijos de largo 1, por lo tanto, un proceso va a entrar y salir de la CPU tantas veces como sea necesario hasta que se termine su tiempo de proceso. Su computador usa el sistema operativo experimental Pikachix el cual implementa una cola con prioridades para almacenar todos los procesos en espera.

Se tienen n procesos, donde cada proceso P_i tiene los siguientes parámetros

- t_i : tiempo total de CPU requerido medido en slots (se ocupa solo 1 por ciclo)
- p_i : prioridad base asignada al proceso(menor valor es mayor prioridad)
- w_i : tiempo de espera en slots

Cada vez que un proceso entra a la cola viene con su prioridad base, tiempo total de CPU definidos. Su tiempo de espera se asigna como el número de procesos de mayor prioridad en la cola y solo cambia cuando pasa por la CPU.

Cada vez que un proceso P_i entra a la CPU su tiempo w_i de espera se recalcula, su tiempo total de CPU requerido disminuye en 1 $t_i \leftarrow t_i - 1$ y la prioridad base disminuye en 1 $p_i \leftarrow p_i + 1$ (note que la prioridad menor es el valor de p_i más alto) y vuelve a la cola.

La prioridad utilizada para ordenar el acceso a la CPU está dada por $prioridad_i = p_i - w_i$

- a) (1,0 pts.) Proponga una estructura de datos para mantener los procesos en espera, indicando los parámetros a almacenar de cada proceso. (asuma que si hay 2 prioridades iguales se usa algun otro parámetro para desempatar)

- b) (3,0 pts.) Cree el pseudocódigo para insertar un nuevo proceso a la estructura, extraer el proceso de mayor prioridad, y actualizar los datos de la estructura cada vez que un proceso es atendido.
- c) (2,0 pts) Indique la complejidad de las operaciones sobre la estructura.

Solución

a) — Estructura de Datos [1.0 pt]

Se propone un Min-Heap (Cola de Prioridad Mínima) donde cada nodo almacena la tupla de un proceso. La prioridad de ordenamiento es: $pe_i = pi - wi$

Campos almacenados por nodo son:

- i Identificador único del proceso
- t_i entero 0 tiempo total de CPU restante (en slots)
- p_i prioridad base actual (menor es más prioritario)
- w_i entero 0 Tiempo de espera (procesos de mayor prioridad en cola)
- pe_i prioridad efectiva entero clave del heap

b) — Pseudocódigo [3.0 pts]

b.1) — Insertar nuevo proceso Al insertar, el tiempo de espera w_i se define como el número de procesos en la cola con prioridad efectiva estrictamente menor al proceso nuevo.

```
insertar(Q, P, t_proc, p_base):
    // Contar procesos con mayor prioridad en la cola
    P.w <- 0 //asignación inicial
    for j=0 to n // recorre todo el heap
        if Q[j].pe < P.p then // wi inicial = 0 provisional
            P.w <- P.w + 1
    // Construir nodo
    P.i <- id // cualquiera único
    P.t <- t_proc
    P.p <- p_base
    P.w <- w_nuevo
    P.pe <- P.p - P.w
    nodo.clave ? nodo.pi ? nodo.wi

    // Insertar en heap y restaurar propiedad
    insertar_en_heap(Q, P)
```

b.2) — $P \leftarrow -extraer(Q)$ Extraer proceso de mayor prioridad es Extraer la raíz del Min-Heap (menor clave = mayor prioridad). Es el mismo algoritmo visto en clases porque al extraer un elemento no se hace ningún cambio sobre los parámetros de los procesos.

b.3) — Actualizar estructura cuando un proceso es atendido por la CPU Cuando el proceso P pasa por la CPU tiene los siguientes efectos: Se extrae el elemento más prioritario del heap $P.p = P.p + 1$ disminuye en 1 $P.w = 0$ $P.t = P.t - 1$ Si $P.t < 0$ el proceso se vuelve a insertar en el heap, de otra forma sale del sistema

```
procesar(Q):
    P <- extract(Q)
    // Actualizar parámetros del proceso atendido
    if p.t > 1 { //el tiempo restante tiene que ser 2 o mas, sino el proceso termina y no reingresa
        P.t <- P.t - 1
        P.p <- P.p + 1
        P.w = 0
        insert(P, Q)
    }
```

c) Complejidad [2,0 pts.]

- insertar $O(n+\log(n))=O(n)$
- extraer $O(\log(n))$ la misma que el heap
- procesar $O(n+\log(n))=O(n)$ básicamente por el insert

IIC2133 Estructuras de Datos y Algoritmos 2026-1

Interrogación 1 – Pauta

5 de mayo de 2026

1. Pregunta 1

Usted está trabajando con una máquina de ordenación que tiene un cabezal de lectura que se mueve sobre un arreglo $A[0..n-1]$. El cabezal siempre está posicionado sobre algún elemento del arreglo. Inicialmente, el cabezal está sobre $A[0]$.

La máquina dispone de las siguientes operaciones primitivas.

- **es-mayor()**, definida cuando el cabezal está sobre una posición i tal que $0 \leq i < n-1$. Retorna **true** si $A[i] > A[i+1]$ y **false** en otro caso. Si $i = n-1$ y se usa esta operación, la máquina se detiene en estado de error.
- **swap()**, definida cuando el cabezal está sobre una posición i tal que $0 \leq i < n-1$. Intercambia los valores de $A[i]$ y $A[i+1]$. Si $i = n-1$ y se usa esta operación, la máquina se detiene en estado de error.
- **avanzar()**, que mueve el cabezal desde la posición actual i hasta la posición $i+1$, siempre que $i < n-1$. Si $i = n-1$ y se usa esta operación, la máquina se detiene en estado de error.
- **retroceder()**, que mueve el cabezal desde la posición actual i hasta la posición $i-1$, siempre que $i > 0$. Si $i = 0$ y se usa esta operación, la máquina se detiene en estado de error.

Cada operación primitiva toma tiempo $O(1)$. Además, cada operación **avanzar** o **retroceder** consume una unidad de energía.

1. (2.0 pts.) Diseñe un algoritmo de ordenamiento basado en esta máquina. Para obtener puntaje completo, su algoritmo debe evitar desperdiciar energía en movimientos del cabezal que no contribuyan al proceso de ordenamiento.

Solución: El siguiente algoritmo recorre el arreglo en ambos sentidos, produciendo intercambios en cada posición en la que dos elementos contiguos sean una inversión. Así, se aprovechan todos los movimientos del cabezal para deshacer inversiones.

Algorithm 1: SORT($A[0..n-1]$)

```
 $\ell \leftarrow 0$ 
 $r \leftarrow n - 1$ 
while  $\ell < r$  do
    // Barrido hacia la derecha desde  $A[\ell]$ .
    for  $i \leftarrow \ell$  to  $r - 1$  do
        if es-mayor() then
            swap()
        if  $i < r - 1$  then
            avanzar()
    // Máximo de  $A[\ell..r]$  quedó en  $A[r]$ , cabezal sobre  $A[r]$ .
     $r \leftarrow r - 1$ 
    if  $\ell \geq r$  then return
    // Barrido hacia la izquierda desde  $A[r]$ .
    for  $i \leftarrow r$  downto  $\ell + 1$  do
        retroceder()
        if es-mayor() then
            swap()
    // Mínimo de  $A[\ell..r]$  quedó en  $A[\ell]$ , cabezal sobre  $A[\ell]$ .
     $\ell \leftarrow \ell + 1$ 
    if  $\ell < r$  then avanzar()
```

Rúbrica de evaluación: Se puede mantener la posición del cabezal usando una variable de tipo entero (la variable i en los dos **for** del algoritmo anterior). No está permitido usar un arreglo adicional, dado que no hay ninguna instrucción de la máquina que permita copiar elementos del arreglo de entrada a otro arreglo. Si implementan un proceso similar al de InsertionSort, el puntaje máximo es 1.5 puntos porque requiere más movimientos de cabezal (el doble en el peor caso, $\approx n^2$ de InsertionSort versus $\approx n^2/2$ del algoritmo propuesto).

2. (1.0 pts) Su algoritmo, ¿es in-place? ¿es estable? Justifique.

Solución: el algoritmo debe ser in-place (dado que no se puede usar un arreglo adicional, sólo $O(1)$ variables adicionales). El algoritmo es estable, ya que nunca intercambia de posición elementos que son iguales: el algoritmo propuesto intercambia sólo si *es-mayor*()).

Rúbrica de Evaluación: Aquí son importantes las justificaciones de las respuestas. Tener en cuenta que es válido si el algoritmo propuesto es no estable.

3. (1.5 pts.) Indique el tiempo de ejecución de su algoritmo en el peor caso, mostrando los pasos del análisis.

Solución: en la primera iteración del **while**, el primer **for** recorre el arreglo desde la posición 0 hasta la $n-1$, ejecutando $n-1$ comparaciones con *es-mayor*() y $n-1$ operaciones *avanzar*()). Luego, el segundo **for** (más la operación *retroceder*()) ejecuta $n-2$ operaciones *retroceder*()).

Análisis del tiempo de ejecución. Sea $m = r - \ell + 1$ el tamaño del intervalo activo al comienzo de una iteración del ciclo **while**. En el primer barrido (primer ciclo **for**), el algoritmo

recorre el intervalo activo de izquierda a derecha, comparando cada par consecutivo

$$A[\ell], A[\ell + 1], \quad A[\ell + 1], A[\ell + 2], \quad \dots, \quad A[r - 1], A[r].$$

Por lo tanto, el primer ciclo **for** ejecuta $m - 1$ llamadas a **es-mayor()**. Además, como el algoritmo no avanza después de la última comparación, ejecuta $m - 2$ operaciones **avanzar()**.

Luego el algoritmo disminuye r en una unidad (porque el máximo del intervalo activo ya quedó en su posición final). Si el algoritmo no termina, el segundo barrido recorre el nuevo intervalo activo de derecha a izquierda. Este nuevo intervalo tiene tamaño $m - 1$, por lo que el segundo ciclo **for** ejecuta $m - 2$ llamadas a **es-mayor()** y $m - 2$ operaciones **retroceder()**.

Finalmente, al terminar el barrido hacia la izquierda, el mínimo del intervalo activo queda en su posición final y el algoritmo incrementa ℓ . Si todavía queda un intervalo activo no trivial, se ejecuta una operación adicional **avanzar()** para dejar el cabezal al comienzo de la siguiente iteración.

Así, en una iteración cuyo intervalo activo tiene tamaño m , el número de llamadas a **es-mayor()** es $(m - 1) + (m - 2) = 2m - 3$. El número de movimientos del cabezal también es $(m - 2) + (m - 2) + 1 = 2m - 3$. Además, cada operación **swap()** ocurre solamente después de una llamada a **es-mayor()**, por lo que el número de swaps es a lo más $2m - 3$ en esa iteración.

Por lo tanto, el costo total de una iteración con intervalo activo de tamaño m es $O(m)$.

Como el intervalo activo disminuye en 2 en cada ciclo **while**, y suponiendo n par, los tamaños de los intervalos activos en cada iteración del ciclo **while** son

$$n, n - 2, n - 4, \dots, 2.$$

Tal como analizamos anteriormente, en una iteración con tamaño de intervalo $m = n - 2j$, para $j = 0, \dots, \frac{n}{2} - 1$, la cantidad de llamadas a **es-mayor()** es $2(n - 2j) - 3$. Por lo tanto, la cantidad total de llamadas a esa operación es

$$T(n) = \sum_{j=0}^{\frac{n}{2}-1} (2(n - 2j) - 3) = \frac{n(n - 1)}{2}.$$

Por lo tanto, $T(n) \in O(n^2)$. Un análisis similar se puede hacer para el resto de las operaciones.

Rúbrica de Evaluación: El análisis no necesita ser exactamente como se muestra anteriormente. Lo importante es que muestre entender que en cada pasada, el número de operaciones es lineal respecto al tamaño actual del intervalo. Si en la parte 1 de esta pregunta respondieron con InsertionSort (o procedimiento similar), considerar que el análisis es distinto pero no descontar puntos si está correcto (debería ser $O(n^2)$ también, pero la constante que multiplica al término cuadrático debería darles mayor que $1/2$).

4. (1.5 pts) Demuestre formalmente que su algoritmo es correcto.

Solución. Para demostrar que el algoritmo es correcto, se deben probar dos cosas: (1) que el algoritmo finaliza en un número finito de pasos (**1/3 del puntaje**), y (2) que el algoritmo cumple su propósito (es decir, ordena, **2/3 del puntaje**).

Finalización. Para demostrar que el algoritmo finaliza en un número finito de pasos, hay que notar que el intervalo activo $[\ell..r]$ se reduce en 2 al finalizar cada iteración del ciclo **while**: notar que r se decrementa en 1 luego del primer ciclo **for**, mientras que ℓ se incrementa en 1 luego del segundo ciclo **for**. Dado que el ciclo **while** itera mientras $\ell < r$ y en cada ciclo ℓ es más grande y r más chico, en un número finito de pasos la condición se hará falsa y el algoritmo finaliza.

Propósito. Para demostrar que el algoritmo ordena, probamos la siguiente invariante usando inducción sobre el número de iteraciones completas del ciclo **while**:

Invariante. Al finalizar la iteración k del ciclo **while**, los k elementos más pequeños del arreglo están en $A[0..k-1]$, ordenados y en sus posiciones finales, y los k elementos más grandes están en $A[n-k..n-1]$, ordenados y en sus posiciones finales.

Caso base. Consideremos $k = 1$, es decir, la situación al finalizar la primera iteración del **while**. En la primera iteración, el primer barrido (primer ciclo **for**) recorre el arreglo desde la izquierda hacia la derecha, intercambiando elementos contiguos del arreglo cuando no están en el orden deseado. Como resultado, después de cada comparación el mayor elemento visto hasta ese momento queda más a la derecha. Por lo tanto, al terminar ese barrido, el máximo de $A[0..n-1]$ queda en $A[n-1]$. Luego el algoritmo reduce r . El segundo barrido (segundo ciclo **for**) recorre el intervalo restante desde la derecha hacia la izquierda. De manera análoga, el menor elemento visto hasta ese momento se va desplazando hacia la izquierda. Por lo tanto, al terminar ese barrido, el mínimo del arreglo queda en $A[0]$.

Así, después de la primera iteración, el menor elemento está en $A[0]$ y el mayor elemento está en $A[n-1]$. Por lo tanto, la invariante se cumple para $k = 1$.

Paso inductivo. Supongamos que la invariante se cumple después de k iteraciones. Entonces, al comenzar la siguiente iteración ($k+1$), los k elementos más pequeños ya están fijos en las primeras k posiciones, y los k elementos más grandes ya están fijos en las últimas k posiciones. Por lo tanto, el único intervalo que puede estar desordenado es $A[k..n-k-1]$.

El barrido hacia la derecha mueve el máximo de $A[k..n-k-1]$ hasta la posición $A[n-k-1]$, por las mismas razones explicadas en el caso base, por lo que ahora los $k+1$ mayores elementos del arreglo queden ordenados en $A[n-k-1..n-1]$. Similarmente, el barrido hacia la izquierda mueve el mínimo del intervalo $A[k..n-k-1]$ a $A[k]$, por lo que ahora $A[0..k]$ contiene los $k+1$ menores elementos de A , ordenados. Los elementos en ambos extremos quedan fijos para el resto del proceso, ya que el algoritmo no vuelve a procesarlos. Luego la invariante se cumple para $k+1$.

Por lo tanto, al terminar el algoritmo, el arreglo completo está ordenado.

Rúbrica de Evaluación: Deben plantear la invariante a demostrar y luego hacerlo por inducción. Si usaron algo tipo InsertionSort, la demostración debe respetar eso.

Dividir para Conquistar

Dado un arreglo $A[0..n-1]$, una inversión es un par (i, j) tal que si $i < j$ entonces $A[i] > A[j]$. La solución directa, comparar cada índice i con todos los j que le siguen, tiene complejidad $O(n^2)$.

- a) [3,0 puntos] Proponga un algoritmo en pseudo código, aplicando la técnica algorítmica Dividir para Conquistar, que permita contar cuantas inversiones existen en el arreglo, con una complejidad de tiempo menor a $O(n^2)$.

Ptos.	Descripción
1.0	La estrategia de solución que sigue los siguientes pasos: 1. Dividir el problema original en sub-problemas del mismo tipo 2. Resolver recursivamente cada sub-problema 3. Encontrar solución al problema original combinando las soluciones a los sub-problema Como la división del problema en mitades (por simplicidad) tiene complejidad $O(\log(n))$, si la cuenta de las inversiones es $O(n)$, entonces la estrategia tiene complejidad $O(n \cdot \log(n))$
1.0	La estructura de la solución es equivalente a MergeSort(), dividiendo el arreglo de entrada hasta tamaño 1. Luego con Merge() contar las inversiones y retornar la suma de las inversiones encontradas. Se retorna la lista ordenada y la cantidad de inversiones detectadas. <pre>MergeSort (A): // I número de inversiones 1 if sizeof(A)= 1: return A,0 // A de tamaño 1 no tiene inversiones 2 Dividir A en mitades A1 y A2 3 B1, I1 ← MergeSort(A1) 4 B2, I2 ← MergeSort(A2) 5 B, I3 ← Merge(B1, B2) 6 return B, I1+I2+I3 // las inv. de este nivel más las llamadas recursivas</pre>
1.0	Para contar inversiones en Merge() notamos que si b es menor que a es una inversión. <pre>Merge(A, B): 1 Iniciamos C vacía 1.1 I = 0 // Cantidad de Inversiones 2 Sean a y b los primeros elementos de A y B 3 Extraer de su secuencia respectiva el menor entre a y b 3.1 if b < a : I ← I + 1 // Encontramos una inversión</pre>

	<pre> 4 Insertar el elemento extraído al final de C 5 Si quedan elementos en A y B, volver a línea 2 6 Concatenar C con la secuencia que aún tenga elementos // No hay más inv. 7 return C, I </pre>
--	--

b) [1,5 puntos] Verifique que su solución es correcta. Indicación: Considere como propiedad a verificar la cantidad de inversiones calculada en el paso (n).

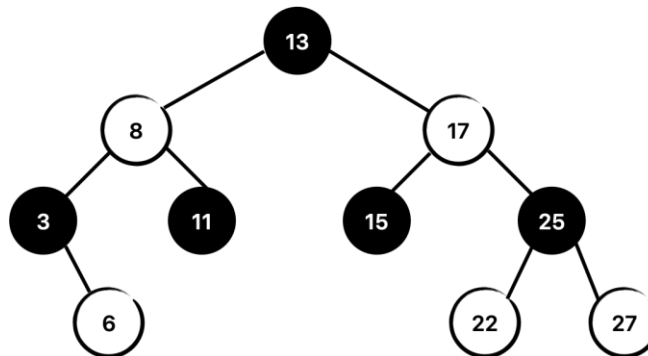
Ptos.	Descripción
0.5	Para ser correcta la solución debe: - Terminar en un número finito de pasos - Realizar correctamente su trabajo
0.5	Termina: - Sabemos que Merge() termina (versión vista en clases), la modificación para contar inversiones no cambia la lógica de ejecución, luego Merge() termina. - De igual forma MergeSort() termina, las divisiones del problema están acotadas a un arreglo de tamaño 1 (con cero inversiones), y luego de hacer Merge() (que termina) retorna cada llamada.
0.5	Realiza su trabajo: En clases se vio que Merge() realiza su trabajo (inducción en el orden de la mezcla), en este caso seguimos la misma lógica para contar las inversiones. Caso Base P(1): Estado de la cuenta al insertar el primer elemento en C. Dos casos: Si a es menor que b no hay inversión. Si b menor que a hay una inversión y se cuenta. H.I. P(n): Luego de insertar el elemento n en C, la cuenta de inversiones es correcta. P.D. P(n+1): Al insertar el elemento n+1 en C, la cuenta de inversiones es correcta. Dos casos: - si el elemento n+1 es extraído de A no suma inversión, la cuenta de inversiones es correcta. - si el elemento n+1 es extraído de B suma inversión, la cuenta de inversiones es correcta. Finalmente, los elementos restantes, de A o B, no suman inversiones. Como sabemos que MergeSort() realiza su trabajo, y la cuenta de inversiones en Merge() es correcta, El algoritmo es correcto.

c) [1,5 puntos] Determine la complejidad $O(?)$ de su solución.

Ptos.	Descripción
0.5	Sabemos que la complejidad de Merge() vista en clases es $O(n)$ con $O(n)$ en memoria. La modificación realizada agrega (línea 3.1) una comparación y asignación $O(1)$, lo que no afecta la complejidad de Merge() que sigue siendo $O(n)$
1.0	Dado que la complejidad de Merge() no cambia, y el resto de la estrategia de solución es MergeSort(), sabemos que la complejidad de la solución es $O(n \cdot \log(n))$

Árboles Rojo-Negro

a) Dado el siguiente árbol Rojo-Negro *ARN* (los nodos sin relleno son rojos)



- i. [1,0 puntos] Inserte los siguientes elementos en dicho árbol mostrando cada uno de los pasos: 14, 10, 9, 4, 2

Ptos.	Descripción
0.2	14: Inserta nodo rojo bajo nodo negro (15): fin
0.2	10: Inserta nodo rojo bajo nodo negro (11): fin
0.2	9: Inserta nodo rojo bajo nodo rojo (10): El tío es negro, inserción exterior, rotación simple y cambio de color. Queda: <pre> ... / R(8) \ N(10) / \ R(9) R(11) </pre>
0.2	4: Inserta nodo rojo bajo nodo rojo (6): El tío es negro, inserción interior, rotación doble y cambio de color. Queda: <pre> ... / R(8) / N(4) / \ R(3) R(6) </pre>

0.2	2: Inserta nodo rojo bajo nodo rojo (3): El tío es rojo (6), cambios de color hacia la raíz. Queda: <pre> N(13) / \ N(8) N(17) / \ ... R(4) N(10) / \ / \ N(3) N(6) ... / R(2) </pre>
-----	---

- ii. [0,5 puntos] Detalle las verificaciones necesarias para comprobar que su resultado es un árbol Rojo-Negro.

Ptos.	Descripción
0.5	Un árbol rojo-negro es un ABB que cumple: 1. Cada nodo es rojo o negro 2. La raíz del árbol es negra 3. Si un nodo es rojo, sus hijos deben ser negros 4. La cantidad de nodos negros camino a cada hoja desde un nodo cualquiera debe ser la misma

- b) Una forma simple de implementar el borrado de una llave k en un árbol Rojo-Negro es “marcar” el nodo x que almacena la llave k como eliminado ($x.deleted = true$), manteniendo la posición del nodo x en el árbol y su color. En este contexto, considerando que el árbol A puede tener nodos eliminados:

- i. [1,0 puntos] Proponga el pseudo código de la búsqueda de la llave k en un árbol Rojo-Negro A : $buscarRN(A, p, k)$

Ptos.	Descripción
0.5	Es básicamente la misma búsqueda de un ABB, pero debe indicar además si el nodo “no encontrado” fue un nodo borrado o no existente (borrado=true o false).
0.5	<pre> buscarRN(A, p, k): 1 if A= null: //no estaba en el árbol 2 return (A, p, false) // si no estaba en el árbol, no estaba borrado </pre>

	<pre> 3 if A.key= k: 4 return (A, p, A.borrado) // Si lo encuentra, indica si estaba borrado 5 if k < A.key: 6 return Search(A.left, A, k) 7 return Search(A.right, A, k) </pre>
--	---

- ii. [1,0 puntos] Proponga el pseudo código de la inserción de la llave k con valor v en un árbol Rojo-Negro A : *insertarRN*(A, k, v)

Ptos.	Descripción
0.5	Es la misma inserción de un ABB, pero debe considerar que el nodo puede no estar en el árbol, estar borrado, o estar vigente.
0.5	<pre> insertarRN (A, k, v): 1 (B, p, borrado) ← buscarRN(A, null, k) // busca desde la raíz 2 if B = null: 3 B ← nodo vacío 4 B.key ← k 5 Conectar B al padre p en la posición adecuada 6 B.value ← v 7 B.borrado ← false 8 FixBalance(B) </pre>

- iii. [0,5 puntos] Proponga el pseudo código de la eliminación de la llave k en un árbol Rojo-Negro A : *borrarRN*(A, k)

Ptos.	Descripción
0.5	Es buscar el nodo en el ABB, si lo encuentra lo marca como borrado. <pre> BorrarRN (A, k): 1 (B, p, borrado) ← buscarRN(A, null, k) // busca desde la raíz 2 if B <> null: 3 B.borrado ← true </pre>

NOTA: Asuma que dispone de la función *FixBalance*(A, x), vista en clases, que recupera la propiedad de árbol Rojo-Negro de A luego de insertar x .

- c) [1,0 puntos] Argumente por qué un árbol Rojo-Negro se considera un *ABB* balanceado, con operaciones con complejidad proporcional al logaritmo del número de nodos del árbol $O(\log(n))$.

Ptos.	Descripción
0.5	Un árbol rojo-negro tiene un árbol 2-4 equivalente, el cual es balanceado con una altura proporcional a $\log(n)$.
0.3	Luego la noción de balance en R-N es equivalente a la de 2-4, por lo que su altura es proporcional a $\log(n)$.
0.2	Como las operaciones tienen complejidad proporcional a la altura del árbol, su complejidad es $O(\log(n))$.

- d) [1,0 puntos] Un problema de “marcar” los nodos borrados en un árbol Rojo-Negro es que luego de un tiempo de uso se puede tener una proporción importante de nodos borrados respecto del número de nodos válidos en el árbol. Proponga (describa) una estrategia simple para “limpiar” el árbol Rojo-Negro, con complejidad no mayor que $O(n*\log(n))$, que se activa cuando el número de nodos borrados es un 20% de los nodos totales almacenados en el árbol. No se requiere el pseudo código.

Ptos.	Descripción
0.5	Una estrategia simple es recorrer en orden el árbol original insertando en un nuevo árbol aquellos nodos que no están borrados.
0.5	La complejidad de recorrer todos los nodos del árbol original es $O(n*\log(n))$. Insertar cada nodo no borrado en el original en el nuevo árbol es $O(n*\log(n))$, la cantidad de nodos no borrados es proporcional a n , y la inserción es $O(\log(n))$. Luego la complejidad total es $O(n*\log(n))$.